



Achieving Efficient and Privacy-Preserving Reverse Skyline Query Over Single Cloud 在单云环境下实现高效且隐私保护的逆向天际查询

汇报人：张睿

2026年6月27日

出处：IEEE TRANSACTIONS ON KNOWLEDGE AND DATA ENGINEERING

目录

C O N T E N T S

01

研究背景

02

理论基础

03

核心贡献

04

性能评估

05

总结思考



南京邮电大学
Nanjing University of Posts and Telecommunications

01

研究背景

南京邮电大学

南京邮电大学

南京邮电大学



1.1 什么是逆向天际查询

天际查询 (Skyline Query) :

帮用户在海量数据中找出“不被别人全面碾压的最好选择”（比如买车时，找那些在价格和性能上没有被其他车全面超越的车型）。

01

逆向天际查询 (Reverse Skyline Query, RSQ) :

视角反过来。它是帮“产品”去找“对它感兴趣的用户”。

02

应用场景：比如一家公司要推一款新产品，公司向存储了历史用户消费数据的云服务器发起 RSQ 请求，找出哪些用户最有可能购买这个新产品，从而进行精准广告推送。

03



1.2 研究背景

逆向天际查询 (Reverse Skyline Query, RSQ) 是一种重要的数据过滤技术，能帮助用户从多维数据集中识别出对其查询向量感兴趣的数据点（例如，广告推送服务中找出潜在客户）。然而，在云计算环境中，数据所有者通常会将数据集外包给云服务器以节省存储和维护成本。这引入了核心挑战：



数据安全与隐私保护

外包到云端的数据容易面临恶意篡改、删除和隐私泄露的风险。同时，用户的查询请求和查询结果也可能暴露其行为模式和个人敏感信息。



单云模型限制

现有的隐私保护 RSQ 方案大多依赖于双云模型（假定两个云服务器互不串通），这在现实中难以实现。



效率低下

现有的隐私保护方案（无论是双云还是单云）往往伴随着高昂的计算和通信开销，使其不适用于实际应用。



本文提出ePRSQ方案，通过设计隐私保护内积符号判定协议(PIPSD)和反向支配检查协议(PRDC)，首次在单云模型下实现了计算高效且隐私保护严格的反向天际线查询。

01 提出PIPSD协议

设计隐私保护内积符号判定方案，通过向量混淆与随机矩阵变换，在不泄露原始向量信息的前提下高效判定内积是否满足特定关系(如 ≤ 0 或 < 0)

02 构建PRDC协议

基于PIPSD和对称同态加密(SHE)，将反向支配判定条件转化为内积计算，实现隐私保护的双向量反向支配关系检查，输出加密结果 $E(1)$ 或 $E(0)$

03 实现ePRSQ完整方案

在单云模型下整合PIPSD与PRDC，通过五阶段协议(初始化、数据外包、查询生成、反向天际线搜索、结果恢复)保护数据集安全、查询请求隐私和查询结果隐私

04 性能优化显著

实验评估显示对于3维1000规模数据集，单次查询计算开销仅79.47秒、通信开销0.0021MB，相比PPARS和OPPRS分别实现3.78倍和61.31倍的计算效率提升



南京邮电大学
Nanjing University of Posts and Telecommunications

02

理论基础

南京邮电大学

南京邮电大学

南京邮电大学



2.1 逆向支配 (Reverse Dominance) 定义

定义 1 (反向支配): 给定两个 d 维数据向量 x_u, x_v 和 q 以及一个 d 维查询向量 x_u , 我们称 q 在反向天空线查询中相对于 x_v 支配 $x_u \prec_{x_v} q$, 记作 $x_u \prec_{x_v} q$, 当且仅当:

$$\forall i \in [1, d] : |x_{ui} - x_{vi}| \leq |q_i - x_{vi}|,$$

and

$$\exists i \in [1, d] : |x_{ui} - x_{vi}| < |q_i - x_{vi}|,$$



2.2 逆向天际查询 (RSQ) 定义

定义 2 (逆天际线查询 (RSQ)): 给定一个包含多个 d 维向量的数据集 χ 和一个查询向量 q , 向量 $x_u \in \chi$ 是 q 的逆天际线向量, 当且仅当不存在 $x_v \in \chi$ 使得 $x_v \prec_{x_u} q$ 。RSQ 返回一个集合 S_q , 其中包含 χ 中所有关于 q 的逆天际线向量, 即 $S_q = \{x_i \in \chi \mid \nexists x_j \in \chi, x_j \prec_{x_i} q\}$ 。

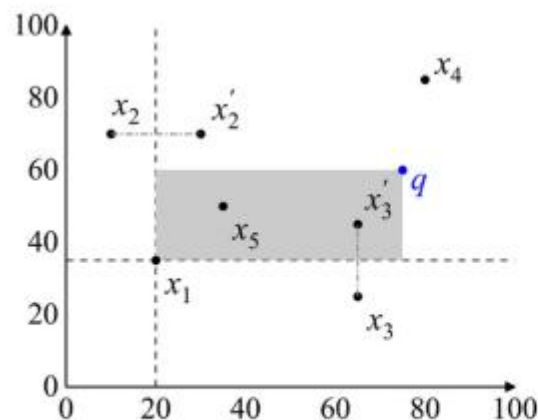
核心: 数据集 X 中 x_u 是 q 的逆向天际向量, 当且仅当 X 中不存在 x_v 逆向支配 q 关于 x_u 。

(a) x_1 不是反向天际线

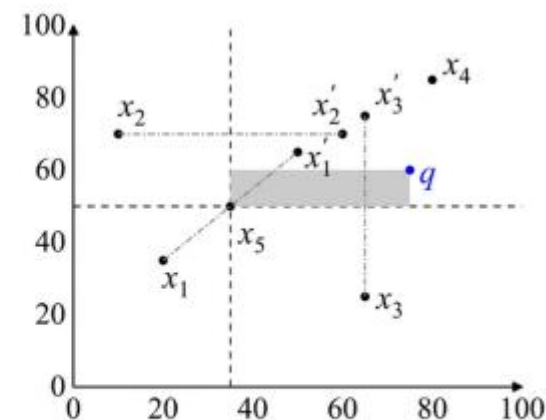
(b) x_5 是反向天际线

图示说明: 以 x_i 为原点构建矩形区域

- 若有其他点落入矩形 $\rightarrow x_i$ 被支配 $\rightarrow$ 不是反向天际线
- 若无点落入矩形 $\rightarrow x_i$ 是反向天际线



(a) x_1 is not a reverse skyline point regarding q



(b) x_5 is a reverse skyline point regarding q



南京邮电大学
Nanjing University of Posts and Telecommunications

2.3 对称同态加密 (Symmetric Homomorphic Encryption, SHE)

概念：

允许在密文上直接执行计算（加法、乘法），而无需解密。

特性

IND-CPA 安全，支持同态加法和乘法。

本文中的作用

本文中的作用：在云端对加密数据进行计算，同时保护数据隐私。



南京邮电大学
Nanjing University of Posts and Telecommunications

03

核心贡献

南京邮电大学

南京邮电大学

南 京 邮 电 大 学

3.1 系统模型

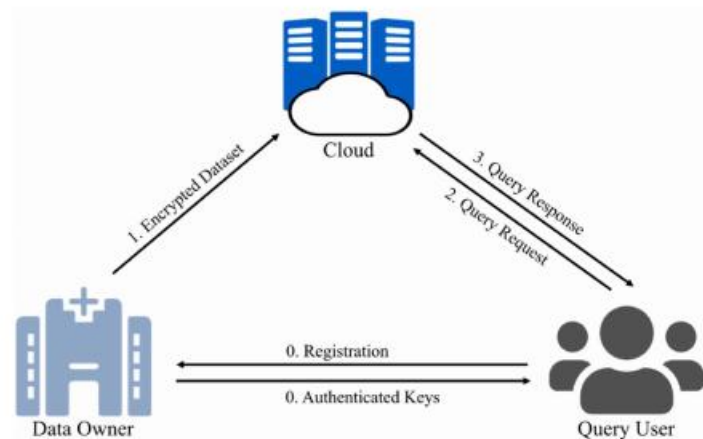


Fig. 2. System Model.

图 2 描绘了多维数据在单个云上进行反向天际线查询的系统模型，该模型包含三个实体：数据所有者 (DO)、云服务器 (CS) 和多个查询用户 (QU)。

DO: 它包含一个 d 维数据集，其中包含 n 个数据向量， $D = \{x_i = (x_{i1}, x_{i2}, \dots, x_{id})\}_{i=1}^n$

。假设数据集中所有值均为整数，这很容易满足。DO 旨在提供反向天际线查询服务，以充分利用数据集。为节省成本，DO 将数据集 D 和相应的 RSQ 服务外包给 CS。为保护数据集 D 的安全，DO 在将其外包给 CS 之前对数据集进行加密。

CS: 云服务器 (Cloud Server, CS) 拥有强大的计算能力和充足的存储资源，负责存储用户 (Data Owner, DO) 的加密数据集，并为查询用户 (Query User, QU) 提供反向天际线查询服务。当 CS 收到 QU 发送的查询请求 Req 时，CS 在数据集 D 中搜索与 Req 对应的反向天际线，并将查询结果密文 S_{sky} 返回给 QU。

QU: 该模型包含一组查询用户。要享受 RSQ 服务，QU 首先需要向 DO 注册。然后，QU 获取授权密钥，并使用该密钥访问反向天际线查询服务。



在安全模型中，我们假设数据所有者（DO）是可信的，这意味着他/她将诚实地外包加密的数据集，并向授权的查询用户（QU）提供有效的私钥。同时，我们假设云服务提供商（CS）是诚实但好奇的，即它会诚实地执行协议，但对其某些私有信息（如数据集的明文、查询请求和查询结果）感到好奇。我们也假设查询用户（QU）是诚实但好奇的，因为QU希望获得正确的查询结果，因此他不会违反协议。然而，QU可能会窃听其他用户的查询请求和结果。我们假设云服务提供商（CS）和查询用户（QU）不共谋。首先，云服务提供商（CS）为了维护其作为云服务提供商的声誉，不与查询用户（QU）共谋是合理的。此外，查询用户（QU）破坏云服务器的成本很高，因为存在法律风险和巨大的经济成本。此外，这种不共谋模型在各种方案中被广泛使用。



3.3 隐私保护的內积符号判定方案 (PIPSD 方案)

动机: RSQ 判定中需要比较內积与0的关系, PIPSD 在不泄露原始向量信息的情况下, 安全地完成这一判定。

核心思想:

数据所有者 (DO) 和查询用户 (QU) 分别对数据进行局部混淆 (Obfuscation)。

云服务器 (CS) 在混淆数据上计算中间结果 (內积), 其符号能反映原始內积与0的关系。

Algorithm 1: Obfuscation Algorithm 1.

Input: PP, x and M .

Output: A $(d+1)$ -dimensional obfuscated vector $\llbracket x \rrbracket_1$.

1: $\alpha = \text{RandomOfBitLength}(K_1)$;

2: $\beta = \text{RandomOfBitLength}(K_2)$;

3: **for** $i \in [1, d]$ **do**

4: $r_i = \text{RandomOfBitLength}(K_0)$;

5: **end for**

6: $\llbracket x \rrbracket_1 = (\alpha x_1 + r_1, \alpha x_2 + r_2, \dots, \alpha x_d + r_d, \beta) \cdot M$

7: **return** $\llbracket x \rrbracket_1$;

Algorithm 2: Obfuscation Algorithm 2.

Input: PP, ∇, y and M^{-1} .

Output: A $(d+1)$ -dimensional obfuscated vector $\llbracket y \rrbracket_2$.

1: $s = \begin{cases} 1, & \nabla \text{ is } > \text{ or } \geq; \\ -1, & \nabla \text{ is } < \text{ or } \leq. \end{cases}$

2: $s' = \begin{cases} 1, & \nabla \text{ is } \leq \text{ or } \geq; \\ -1, & \nabla \text{ is } < \text{ or } >. \end{cases}$

3: $\alpha' = \text{RandomOfBitLength}(K_1)$

4: $\beta' = \text{RandomOfBitLength}(K_2)$;

5: **for** i in $[1, d]$ **do**

6: $r'_i = \text{RandomOfBitLength}(K_0)$;

7: **end for**

8: $\llbracket y \rrbracket_2 =$

$M^{-1} \cdot (s\alpha'y_1 + r'_1, s\alpha'y_2 + r'_2, \dots, s\alpha'y_d + r'_d, s'\beta')^T$

9: **return** $\llbracket y \rrbracket_2$;



3.3 隐私保护的內积符号判定方案 (PIPSD 方案)

Algorithm 1: Obfuscation Algorithm 1.

Input: PP, x and M .

Output: A $(d+1)$ -dimensional obfuscated vector $\llbracket x \rrbracket_1$.

```
1:  $\alpha = \text{RandomOfBitLength}(K_1)$ ;  
2:  $\beta = \text{RandomOfBitLength}(K_2)$ ;  
3: for  $i \in [1, d]$  do  
4:    $r_i = \text{RandomOfBitLength}(K_0)$ ;  
5: end for  
6:  $\llbracket x \rrbracket_1 = (\alpha x_1 + r_1, \alpha x_2 + r_2, \dots, \alpha x_d + r_d, \beta) \cdot M$   
7: return  $\llbracket x \rrbracket_1$ ;
```

Algorithm 2: Obfuscation Algorithm 2.

Input: PP, ∇, y and M^{-1} .

Output: A $(d+1)$ -dimensional obfuscated vector $\llbracket y \rrbracket_2$.

```
1:  $s = \begin{cases} 1, & \nabla \text{ is } > \text{ or } \geq; \\ -1, & \nabla \text{ is } < \text{ or } \leq. \end{cases}$   
2:  $s' = \begin{cases} 1, & \nabla \text{ is } \leq \text{ or } \geq; \\ -1, & \nabla \text{ is } < \text{ or } >. \end{cases}$   
3:  $\alpha' = \text{RandomOfBitLength}(K_1)$   
4:  $\beta' = \text{RandomOfBitLength}(K_2)$ ;  
5: for  $i$  in  $[1, d]$  do  
6:    $r'_i = \text{RandomOfBitLength}(K_0)$ ;  
7: end for  
8:  $\llbracket y \rrbracket_2 =$   
    $M^{-1} \cdot (s\alpha'y_1 + r'_1, s\alpha'y_2 + r'_2, \dots, s\alpha'y_d + r'_d, s'\beta')^T$   
9: return  $\llbracket y \rrbracket_2$ ;
```

工作机制：

设置：生成秘密可逆矩阵 M 。

混淆算法 Obf1 (DO):

输入：数据向量 x ，秘密矩阵 M 。

过程： x 通过随机数和 M 矩阵进行转换，得到混淆向量 $\llbracket x \rrbracket_1$ 。

作用：隐藏 x 的明文。

混淆算法 Obf2 (QU):

输入：查询向量 y ，秘密矩阵 M^{-1} ，关系符号 ∇ 。

过程： y 通过符号因子、随机数和 M^{-1} 矩阵转换，得到混淆向量 $\llbracket y \rrbracket_2$ 。

作用：隐藏 y 的明文，并引入符号因子以关联内积关系。



3.3 隐私保护的內积符号判定方案 (PIPSD 方案)

TABLE III
RELATIONSHIP BETWEEN ∇ , $\langle X, y \rangle$ AND δ

∇	s	s'	$\langle x, y \rangle$	δ
$>$	1	-1	> 0	> 0
			$= 0$	< 0
			< 0	< 0
$\geq$	1	1	> 0	> 0
			$= 0$	> 0
			< 0	< 0
$<$	-1	-1	> 0	< 0
			$= 0$	< 0
			< 0	> 0
$\leq$	-1	1	> 0	< 0
			$= 0$	> 0
			< 0	> 0

表 III ∇ , $\langle X, y \rangle$ 与 δ 之间的关系

检查 (Check - CS):

计算 $\delta = \langle [x]_1, [y]_2 \rangle$ 。

δ 的符号 (>0 或 <0) 直接决定 $(x, y) \nabla 0$ 是否成立。

PIPSD 方案的设计确保了, 通过 QU 巧妙地选择 s 和 s' , 云服务器仅仅通过判断 δ 的符号, 就能间接地、隐私地推断出真实內积 $\langle x, y \rangle$ 是否满足预设的关系 ∇ 。



3.4 PRDC方案

基于PIPSD，本文设计了一个隐私保护的反支配性检查方案，记为PRDC。它可以以隐私保护的方式检查两个数据之间是否存在反支配性关系。

具体来说，给定两个d维数据向量 x_u 、 x_v 和一个查询向量 q ，PRDC可以在不泄露原始向量信息的情况下确定 $x_u <_{xv} q$ 是否成立。如果成立，则输出SHE密文 $E(\tau) = E(1)$ 。否则，输出 $E(\tau) = E(0)$ 。

核心思想：将逆向支配的两个条件转换为多个内积与0的关系判定问题，然后利用PIPSD和SHE来安全地解决。

Algorithm 3: PRDC Encryption Algorithm.

Input: PP, M_1, M_2, x_u and x_v .

Output: $[\chi(u, v)]^*$.

- 1: $\chi(u, v) = (x_{u1}^2 - 2x_{u1}x_{v1}, \dots, x_{ud}^2 - 2x_{ud}x_{vd}, x_{v1}, \dots, x_{vd}, 1)$
- 2: $[\chi(u, v)]_1 = PIPSD.Obf_1(PP, \chi(u, v), M_1)$
- 3: $[\chi(u, v)]^* = [\chi(u, v)]_1 \cdot M_2$
- 4: **return** $[\chi(u, v)]^*$;

Algorithm 4: PRDC Query Generation.

Input: $PP, M_1^{-1}, M_2^{-1}, q$ and pk .

Output: A query token q_t .

- 1: **for** $i \in [1, d]$ **do**
- 2: $\eta(i)_{j \in [1, 2d+1]} = \begin{cases} 1, & j = i; \\ 2q_i, & j = d + i; \\ -q_i^2, & j = 2d + 1; \\ 0, & \text{Otherwise.} \end{cases}$
- 3: $[\eta(i)]_2 = PIPSD.Obf_2(PP, \leq, \eta(i), M_1^{-1})$;
- 4: Select random number $s_i \in \{-1, 1\}$;
- 5: $E(\mu_i) = \begin{cases} E(0), & s_i = 1; \\ E(1), & s_i = -1. \end{cases}$
- 6: **end for**
- 7: $\gamma = (1, \dots, 1, 2q_1, \dots, 2q_d, -\sum_{i=1}^d q_i^2)$;
- 8: $[\gamma]_2 = PIPSD.Obf_2(PP, <, \gamma, M_1^{-1})$;
- 9: $\psi = M_2^{-1} \cdot (s_1[\eta(1)]_2^T, \dots, s_d[\eta(d)]_2^T, s_{d+1}[\gamma]_2^T)$;
- 10: $q_t = \{\psi, \{E(\mu_i)\}_{i=1}^{d+1}\}$
- 11: **return** q_t ;

Algorithm 5: PRDC Checking Algorithm.

Input: $[\chi(u, v)]^*, q_t$ and pk .

Output: An encrypted check result $E(\tau)$.

- 1: $\Delta = [\chi(u, v)]^* \cdot \psi$
- 2: **for** $i \in [1, d]$ **do**
- 3: $E(\delta_i) = \begin{cases} E(1) - E(\mu_i), & \Delta_i > 0; \\ E(\mu_i), & \Delta_i < 0. \end{cases}$
- 4: **end for**
- 5: $E(\tau) = \prod_{i=1}^{d+1} E(\delta_i)$;
- 6: **return** $E(\tau)$;



3.4 PRDC方案

Algorithm 3: PRDC Encryption Algorithm.

Input: PP, M_1, M_2, x_u and x_v .

Output: $[\chi(u, v)]^*$.

```

1:  $\chi(u, v) =$ 
    $(x_{u1}^2 - 2x_{u1}x_{v1}, \dots, x_{ud}^2 - 2x_{ud}x_{vd}, x_{v1}, \dots, x_{vd}, 1)$ 
2:  $[\chi(u, v)]_1 = PIPSD.Obf_1(PP, \chi(u, v), M_1)$ 
3:  $[\chi(u, v)]^* = [\chi(u, v)]_1 \cdot M_2$ 
4: return  $[\chi(u, v)]^*$ ;

```

Algorithm 4: PRDC Query Generation.

Input: $PP, M_1^{-1}, M_2^{-1}, q$ and pk .

Output: A query token q_t .

```

1: for  $i \in [1, d]$  do
2:    $\eta(i)_{j \in [1, 2d+1]} = \begin{cases} 1, & j = i; \\ 2q_i, & j = d + i; \\ -q_i^2, & j = 2d + 1; \\ 0, & \text{Otherwise.} \end{cases}$ 
3:    $[\eta(i)]_2 = PIPSD.Obf_2(PP, \leq, \eta(i), M_1^{-1});$ 
4:   Select random number  $s_i \in \{-1, 1\}$ ;
5:    $E(\mu_i) = \begin{cases} E(0), & s_i = 1; \\ E(1), & s_i = -1. \end{cases}$ 
6: end for
7:  $\gamma = (1, \dots, 1, 2q_1, \dots, 2q_d, -\sum_{i=1}^d q_i^2);$ 
8:  $[\gamma]_2 = PIPSD.Obf_2(PP, <, \gamma, M_1^{-1});$ 
9:  $\psi = M_2^{-1} \cdot (s_1[\eta(1)]_2^T, \dots, s_d[\eta(d)]_2^T, s_{d+1}[\gamma]_2^T);$ 
10:  $q_t = \{\psi, \{E(\mu_i)\}_{i=1}^{d+1}\}$ 
11: return  $q_t$ ;

```

PRDC方案包含四个算法:

1) 设置(d, T): 给定向量在 d 维度的维度 d 和向量元素绝对值的上限 T, 算法输出公有参数

2) Enc(x_u, x_v, PP, M_1, M_2): 给定任意两个d维数据向量 x_u 和 x_v 、公钥参数PP以及两个秘密矩阵M1和M2, 加密算法为 x_u 和 x_v 生成一个 $(2d + 2)$ 维的混淆向量 $[\chi(u, v)]^*$ 。加密算法详见算法3。

3)

QueryGen($q, PP, M_1^{-1}, M_2^{-1}, pk$):

给定一个d维查询向量q, 公钥参数PP, 两个秘密矩阵 M_1^{-1}, M_2^{-1} 以及SHE的公钥参数pk, 查询生成算法(如算法4所示) 输出一个查询令牌 q_t 。



Algorithm 5: PRDC Checking Algorithm.

Input: $[\chi(u, v)]^*$, q_t and pk .

Output: An encrypted check result $E(\tau)$.

1: $\Delta = [\chi(u, v)]^* \cdot \psi$

2: **for** $i \in [1, d]$ **do**

3: $E(\delta_i) = \begin{cases} E(1) - E(\mu_i), & \Delta_i > 0; \\ E(\mu_i), & \Delta_i < 0. \end{cases}$

4: **end for**

5: $E(\tau) = \prod_{i=1}^{d+1} E(\delta_i)$;

6: **return** $E(\tau)$;

4) Check($[\chi(u, v)]^*$, q_t , pk):

给定SHE的公钥参数 pk 、一个混淆向量 $[\chi(u, v)]^*$ 和一个查询令牌 q_t ，检查算法输出一个加密的检查结果 $E(\tau)$ ，其中 $E(\tau) = E(1)$ 如果 $x_u < x_v$ q 成立，否则 $E(\tau) = E(0)$ 。该算法在算法5中给出。

本文的 ePRSQ 方案包含以下五个阶段。

1) 系统初始化：假设数据所有者 (DO) 拥有一个 d 维数据集 D ，其中数据绝对值的上界为 T 。为了初始化系统，DO 首先选择一个非对称加密算法 $AE(\cdot)$ ，例如 ElGamal，并随机生成密钥对 $\{sk_{AE}, pk_{AE}\}$ 。然后，DO 生成 PRDC 的参数，即 $\{P, P, M1, M2\} = P, RDC.Setup(d, T)$ 。最后，DO 发布公有参数 $\{P, P, pk_{AE}\}$ ，并通过安全通道将 sk_{AE} 发送给认证中心 (CS)。同时，如果查询方 (QU) 想使用 RSQ 服务，他/她必须首先向 DO 注册。经 DO 批准后， $M1^{-1}$ 和 $M2^{-1}$ 将通过安全通信通道传输给 QU。

2) 数据外包：在此阶段，DO 首先使用算法 6 将数据集 D 转换为加密数据集 $[[D]]$ ，以保证数据集 D 的安全性。之后，DO 将 $[[D]]$ 外包给 CS。

3) 查询生成：首先，对于查询向量 q ，QU 根据 PRDC 查询生成算法生成查询令牌 $qt = P, RDC.QueryGen(q, P, P, M1^{-1}, M2^{-1}, pk)$ 。其次，QU 使用 CS 的公钥 pk_{AE} 加密查询令牌 qt ，即 $AE(pk_{AE}, qt)$ ，并选择 SHE 参数 $\{pk, sk\}$ 。最后，QU 将查询请求 $Req = \{AE(pk_{AE}, qt), pk\}$ 发送给 CS 并秘密保留 sk 。

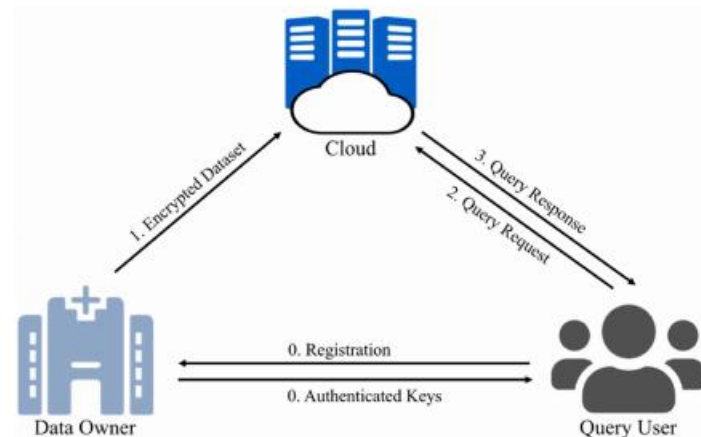


Fig. 2. System Model.



本文的 ePRSQ 方案包含以下五个阶段。

4) 反向天际线搜索：一旦CS收到QU的查询请求Req，它会提取pk并使用skAE解密AE(pkAE, qt)以恢复查询令牌qt。然后，CS执行算法7以执行隐私保护的反向天际线查询。最后，CS将加密的查询结果Ssky发送给QU。

5) 结果恢复：在接收到CS发送的加密查询结果后，QU执行算法8以恢复关于q的反向天际线ID。

Algorithm 7: ePRSQ Reverse Skyline Search Algorithm.

Input: An encrypted dataset

$\{\llbracket \chi(i, j) \rrbracket^* \mid i, j \in [1, n] \wedge i \neq j\}$, a query token q_t , and a SHE public parameter pk .

Output: A set containing n data indexes and SHE ciphertexts, $S_{sky} = \{i \parallel E(\theta_i) \mid 1 \leq i \leq n\}$.

```

1:  $S_{sky} = \emptyset$ ;
2:  $i = 1$ ;
3: while  $i \leq n$  do
4:    $j = 1$ ;
5:    $E(\theta_i) = E(0)$ ;
6:   while  $j \leq n$  do
7:      $E(\tau) = PRDC.Check(\llbracket \chi(j, i) \rrbracket^*, q_t, pk)$ ;
8:      $E(\theta_i) = E(\theta_i) + E(\tau)$ ;
9:      $j = j + 1$ ;
10:  end while
11:   $S_{sky} \leftarrow (i \parallel E(\theta_i))$ ;
12:   $i = i + 1$ ;
13: end while
14: return  $S_{sky}$ ;

```

Algorithm 8: ePRSQ Query Result Recover.

Input: An encrypted query result

$S_{sky} = \{i \parallel E(\theta_i) \mid 1 \leq i \leq n\}$, and a SHE private parameter sk .

Output: A set S_{ID} that contains reverse skylines' IDs.

```

1:  $S_{ID} = \emptyset$ ;
2: for  $i \parallel E(\theta_i)$  in  $S_{sky}$  do
3:    $\theta_i = SHE.Dec(sk, E(\theta_i))$ ;
4:   if  $\theta_i = 0$  then
5:      $S_{ID} \leftarrow i$ ;
6:   end if
7: end for
8: return  $S_{ID}$ ;

```



南京邮电大学
Nanjing University of Posts and Telecommunications

04

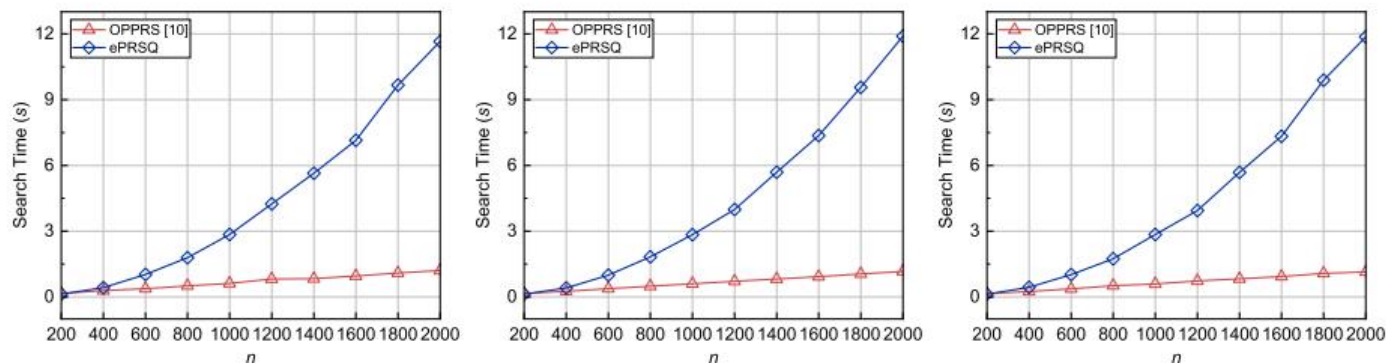
性能评估

南京邮电大学

南京邮电大学

南 京 邮 电 大 学

4.1 数据外包阶段

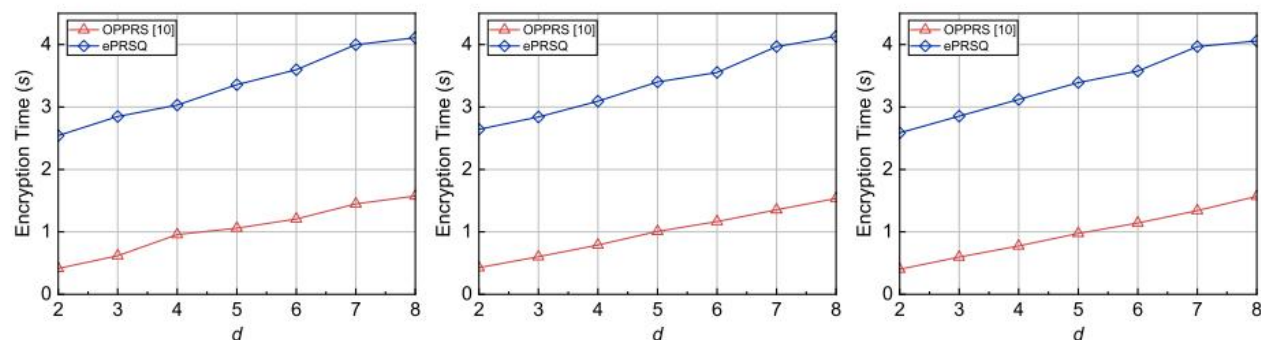


(a) EEG Eye State dataset [36]

(b) India House dataset [37]

(c) US Census dataset [38]

图 3. 不同数据集上离线数据加密的计算成本, 该成本随 n 和 $d = 3$ 的变化而变化。



(a) EEG Eye State dataset [36]

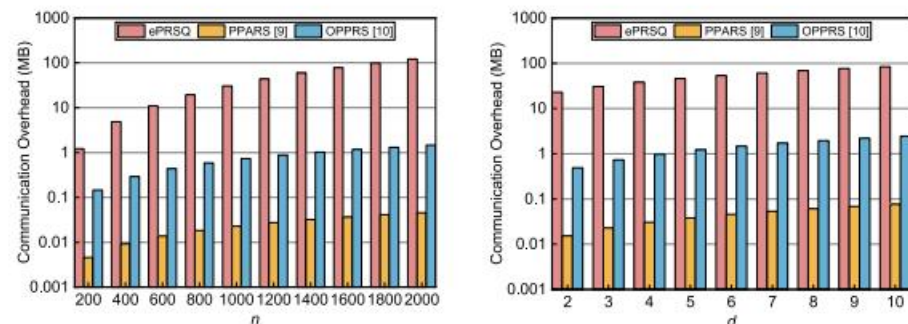
(b) India-house dataset [37]

(c) US-census dataset [38]

图 4. 不同数据集上离线数据加密的计算成本, 该成本随 d 和 $n = 1000$ 而变化。

计算成本 (Fig. 3 & 4): ePRSQ 略高于对比方案, 但此阶段是一次性、离线操作, 可接受。

通信开销 (Fig. 8): ePRSQ 相对较高, 但同样是一次性开销, 且保护了数据隐私。



(a) Data outsourcing phase where $d = 3$ and n changes

(b) Data outsourcing phase where $n = 1000$ and d changes

图 8. 数据外包的通信开销。



4.2 查询生成阶段

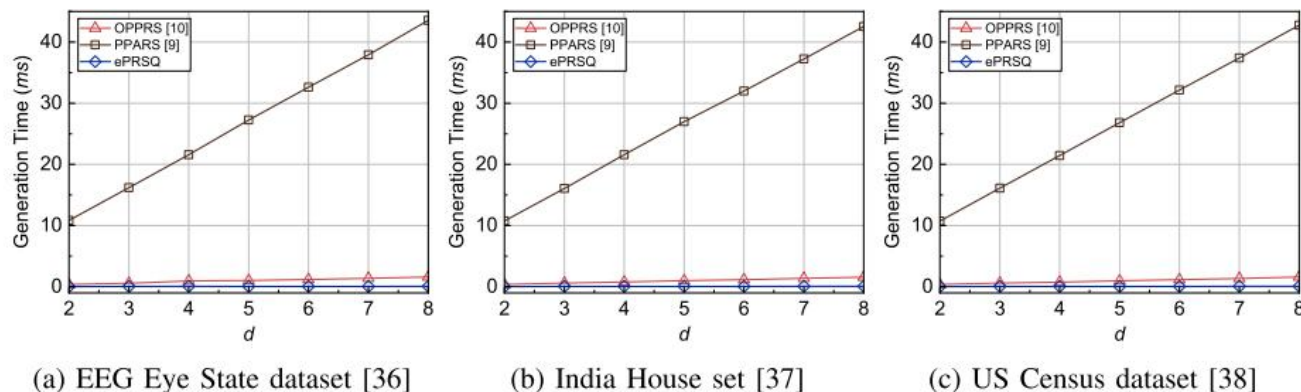


图 5. 查询生成在不同数据集上的计算成本随 d 和 $n = 1000$ 的变化。

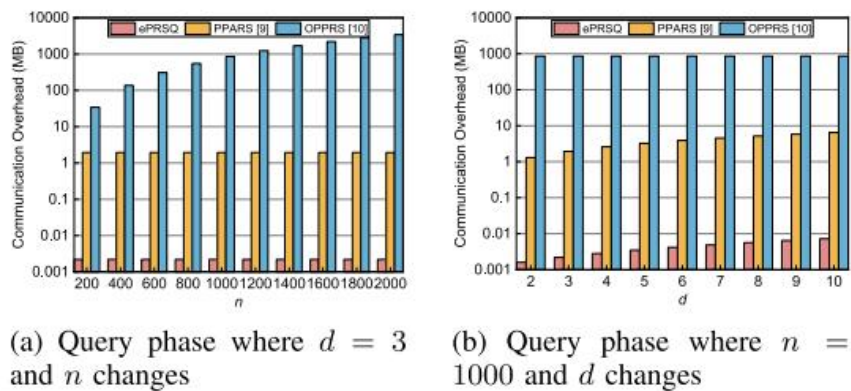


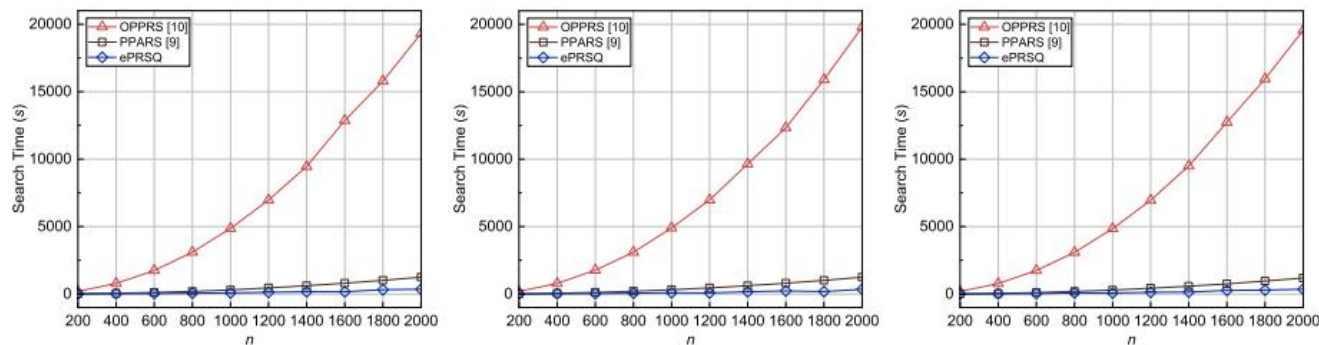
图 9. 查询反向天际线的通信开销。

计算成本 (Fig. 5): ePRSQ 显著优于 PPARS 和 OPPRS (快 20-500 倍)。

通信开销 (Fig. 9(b)): ePRSQ 远小于 PPARS 和 OPPRS (尤其当 d 较小时)。

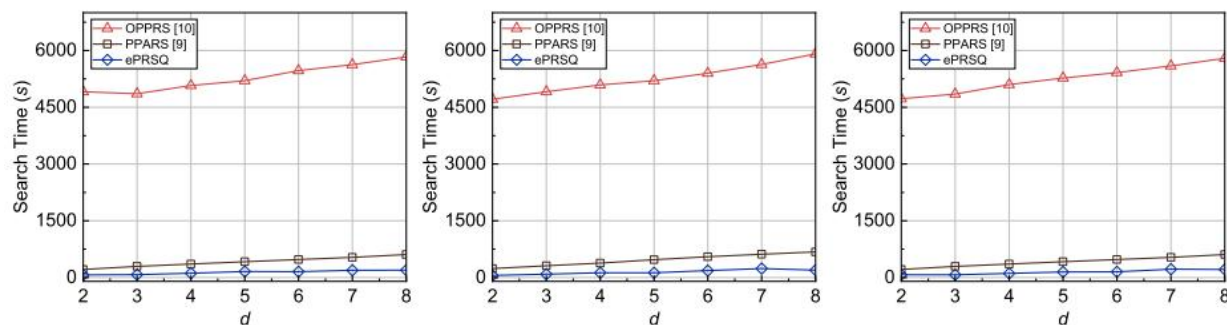


4.3 逆向天际搜索阶段



(a) EEG Eye State dataset [36] (b) India House dataset [37] (c) US Census dataset [38]

图 6. 在不同数据集上搜索的计算成本随 n 和 $d = 3$ 的变化情况。

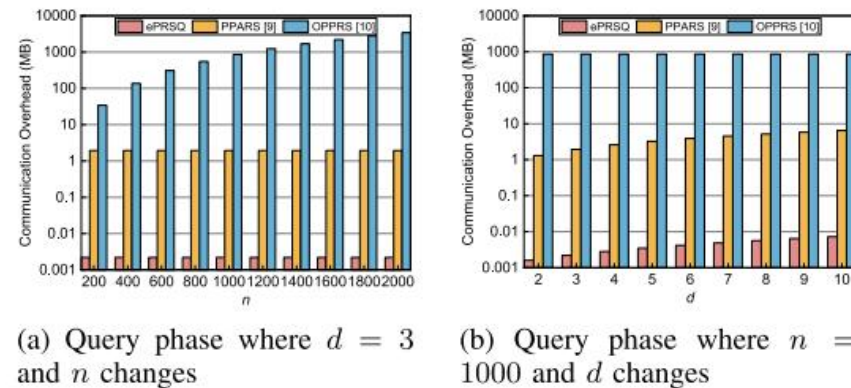


(a) EEG Eye State dataset [36] (b) India House dataset [37] (c) US Census dataset [38]

图 7. 在不同数据集上搜索的计算成本随 d 和 $n = 1000$ 的变化。

计算成本 (Fig. 6 & 7): ePRSQ 显著优于 PPARS 和 OPPRS (快 3-60 倍)。

通信开销 (Fig. 9(a)): ePRSQ 优势巨大 (比 PPARS 少 300 倍, 比 OPPRS 少 40 万倍)。。



(a) Query phase where $d = 3$ and n changes

(b) Query phase where $n = 1000$ and d changes

图 9. 查询反向天际线的通信开销。



4.4 文献复现结果

Dataset	Experiment	d	n	加密_Time_s	Query_Gen_Time_s	Search_Time_s	Storage_MB	Key_Gen_Time_s	Data_outsourcing
EyeState	Exp1_Vary	3	200	0.7727	19.4038	10.3859	19.4852	0.003	0.7757
EyeState	Exp1_Vary	3	400	3.7814	73.2085	42.4773	78.6145	0.0021	3.7835
EyeState	Exp1_Vary	3	600	5.1354	28.4911	93.8795	175.9934	0.002	5.1374
EyeState	Exp1_Vary	3	800	6.7297	34.3508	167.5686	314.8858	0.0018	6.7315
EyeState	Exp1_Vary	3	1000	9.4782	25.4108	260.7035	484.1456	0.0022	9.4804
EyeState	Exp1_Vary	3	1200	14.9903	35.3838	381.5314	711.6855	0.002	14.9923
EyeState	Exp1_Vary	3	1400	21.9143	54.1133	515.1744	963.207	0.0013	21.9156
EyeState	Exp1_Vary	3	1600	29.9818	4.5448	676.0031	1249.8671	0.002	29.9838
EyeState	Exp1_Vary	3	1800	41.0213	10.7993	846.1374	1579.5539	0.001	41.0223
EyeState	Exp1_Vary	3	1999	59.5694	87.6002	1055.8188	1950.3282	0.0025	59.5719
EyeState	Exp2_Vary	2	1000	6.3832	19.3096	179.0616	378.23	0.0022	6.3854
EyeState	Exp2_Vary	3	1000	10.0481	23.6344	264.3864	491.3508	0.0021	10.0502
EyeState	Exp2_Vary	4	1000	10.3037	60.514	354.3996	596.2737	0.0021	10.3058
EyeState	Exp2_Vary	5	1000	11.2861	69.6285	435.9259	709.6445	0.0025	11.2886
EyeState	Exp2_Vary	6	1000	12.3776	80.9505	530.9246	826.17	0.0023	12.3799
EyeState	Exp2_Vary	7	1000	13.6622	38.3183	619.6667	932.8741	0.003	13.6652
EyeState	Exp2_Vary	8	1000	15.3769	27.2628	711.3853	1045.5477	0.004	15.3809

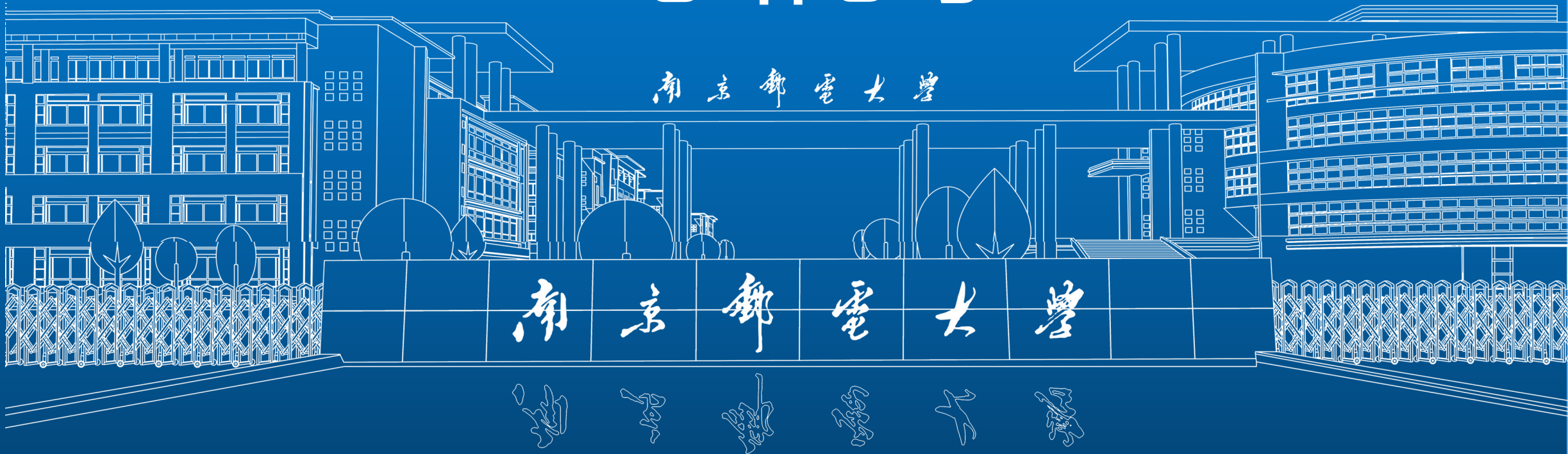
通过分析复现结果可以看出，本文的实验部分避重就轻的讲了方案的通信开销，但没有讲因为其协议设计，导致存储开销过大。且查询时间只是相对于其baseline有所提升，但本身在n=2000大小的数据集上查询时间高达数百秒，效果依然很差。



南京邮电大学
Nanjing University of Posts and Telecommunications

05

总结思考





5.1 总结思考

本文总结：

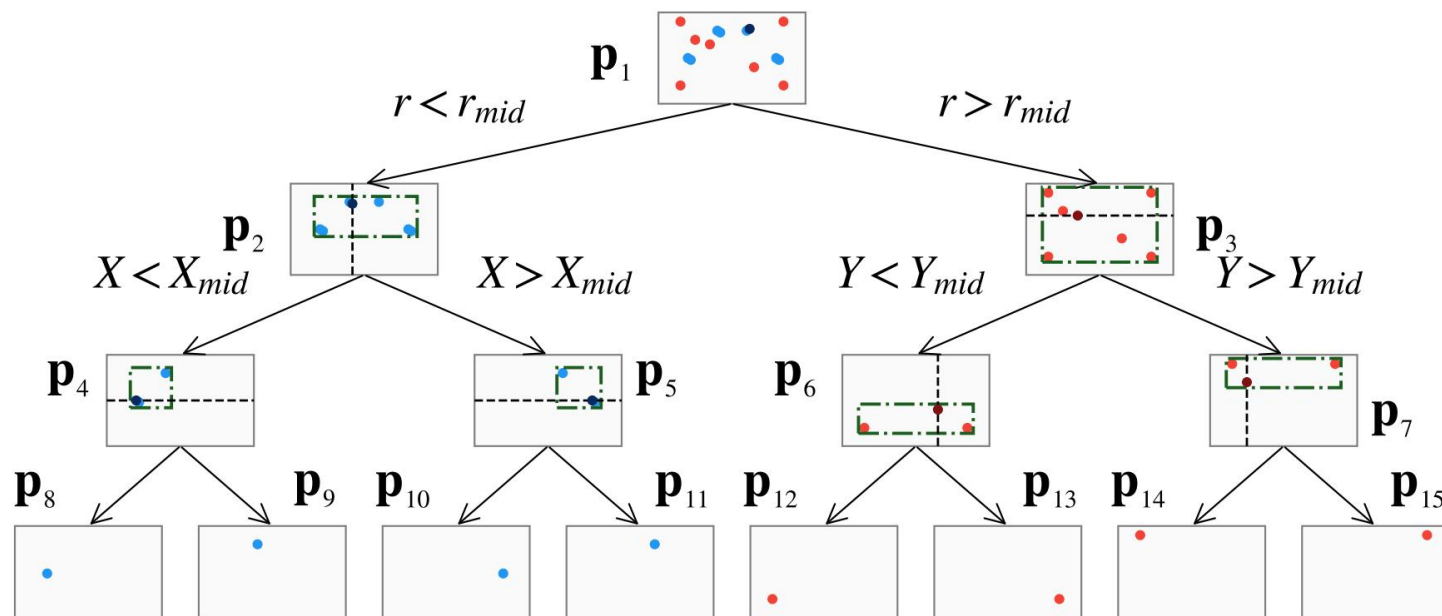
针对单云RSQ的隐私保护和效率问题。

提出了 PIPSD 和 PRDC 两个隐私保护协议。

基于此构建了高效隐私保护的单云 RSQ 方案 (ePRSQ)。

但本文为了实现高隐私保护的需求没有设计任何索引，时间复杂度依旧为 $O(n^2)$ 。

改进方向：通过利用空间性质，构造kd树索引，对原始数据进行剪枝，牺牲部分安全性换取效率提升





南京邮电大学

Nanjing University of Posts and Telecommunications

感谢观看

Thanks

汇报人：张睿

2026.6.27